
Lab W03: A Recursive Multi-Level Cache Simulator

CIS 4020 / CS 2100, Data Structures

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

1 Background

Modern processors do not read every value straight from main memory. Main memory is large but slow compared to the processor, so a chain of smaller, faster caches sits between the processor and memory. When the processor asks for a piece of data, it checks the fastest cache first. If the data is there, the request is answered right away. If not, the check moves to the next cache in the chain, which is a little larger and a little slower, and so on, until the data is found or the request finally reaches main memory, which always has it.

This chain of checks, one level at a time, each one asking the next level for help when it comes up empty, is a natural fit for recursion. That is the connection this lab asks you to build. You will not need any background in computer architecture beyond what is written here. The lab is about writing a correct recursive function, using the cache chain as the setting.

2 The model

The simulation you will build works as follows.

- There are N cache levels, numbered from level 1 (fastest, checked first) up to level N . Beyond level N comes main memory, which is not really a cache. It always has the data, but it is the slowest place to get it from.
- Each cache level has two properties: a capacity, which is the number of distinct blocks it can hold, and a hit time, which is the number of cycles it costs to check that level.
- Main memory has a single access time, in cycles, and unlimited capacity.
- All caches start empty. A trace of memory accesses, a list of block IDs in the order they are requested, is fed to your program one block at a time.
- To service a request, the simulator checks level 1. If the block is there, that level answers the request and the cost is that level's hit time. If not, the request moves to level 2, and so on, until either some level answers it or the request falls through to main memory.

-
- After a lower level (or main memory) answers a request, the block is copied into the level that missed, but only if that level still has room. This lab does not implement eviction. If a level is full, a missed block is simply not stored there, and that level stays as it was.
 - The total cost of a request is the sum of the hit times of every level that was checked, plus the main memory access time if the request reached main memory.

A real cache almost always needs a way to make room when a level is full and a new block needs to be stored there. This is called eviction. A real cache picks an existing block to remove, usually based on some replacement policy such as removing whichever block was used least recently, and puts the new block in its place. This lab does not ask you to implement any of that. Once a level in this simulation fills up, it simply stops accepting new blocks for the rest of the run. It is worth knowing that eviction exists and why real systems need it, since it explains why cache design in practice is more involved than the version you are building here, but you do not need to write any code for it.

Something to think about (not graded)

In the sample trace later in this handout, block 5 is requested a second time and becomes much faster. Block 9 is also requested a second time, and it becomes faster too, but not by as much as block 5 did. Once you have your program running, look at where each of those two blocks ended up being cached, and think about why the capacity of level 1 matters here. You do not need to write anything down. Just be ready to explain it if asked.

3 Why this has to be recursion

Write a single recursive function that decides how one request is serviced. Call it something like `accessLevel(blockID, level)`, and have it return the cost, in cycles, of servicing that block starting its search at the given level. Loosely, the function has three cases.

```
accessLevel(blockID, level):
    if level is past the last cache level:
        this request is served by main memory
        return the main memory access time

    if blockID is already cached at this level:
        this request is served by this level
        return this level's hit time

    otherwise:
        ask the next level down what it costs to get this block
        add this level's hit time to that answer
        if this level still has room, cache the block here now
        return the combined cost
```

The part worth paying attention to is the last case. The work of caching the block only makes sense once you already know the block was found somewhere below, which means it has to happen after the recursive call returns, not before it. That "do something after the call comes back" is the part of recursion that trips people up the most, and it is also the part that mirrors how the cascading lookup through the cache levels is normally analyzed. Real hardware does not use a software call stack to do this. Comparator circuits and state machines handle the actual lookups, and multiple levels are often checked in parallel rather than strictly one after another. The recursive

shape you are writing matches the logic of the lookup and the math used to analyze it, not the physical circuitry.

You may use loops elsewhere in your program, for example to read the input or to print the summary at the end. The chain of checks from level 1 down to main memory must be done with recursion, not with a loop.

4 Requirements

- Name your project `labw03`, following the project organization discussed in the Course Information module.
- If you are using Java, write your solution in a single file named `MultiLevelCache.java`.
- If you are using C or C++, write your solution in a single file named `multilevel_cache.cpp`.
- Your program must read its input from a file named `input.txt` in your project's working directory, and write its output to standard output (the console), in the exact formats described below.
- The chain of level-by-level checks must be implemented as a recursive function. Do not replace it with a loop or a while-based chain of checks.
- Do not implement any form of cache eviction. If a level is full when a block would be inserted, skip the insertion.
- For a typical solution, `accessLevel` is the only method you should need to write or change. The starter file already handles reading the input, printing each access, and printing the summary.
- Every source file you submit needs the general comments described in the syllabus (4.10.8 General Comments): at least one sentence describing the class, the assignment title, a version number (starting at 1.0 is fine), and your name as the author. The starter file already has a comment block in this form near the top; fill in your name and adjust the description sentence if you like, rather than writing this from scratch.

5 Compiling and running (C/C++ students)

From a terminal in the same directory as your `multilevel_cache.cpp` file, compile with:

```
g++ -std=c++17 -o multilevel_cache multilevel_cache.cpp
```

The `-std=c++17` flag tells the compiler which version of the C++ language standard to use. This code does not require any C++17-specific features, so compiling without that flag will also work, but it is a common default in course toolchains and is a safe choice to include.

Then run the resulting program, with `input.txt` present in that same directory:

```
./multilevel_cache
```

If you are working in an IDE instead, use your IDE's normal build and run commands. Just make sure `input.txt` ends up in the working directory your program runs from.

6 Input format

Your program should read the input described below from a file named `input.txt`, located in your project's working directory, rather than from the console.

Table 1: Input format, one item per line unless noted

Line(s)	Contents
1	N , the number of cache levels
2 through $N + 1$	For each level, in order from level 1 to level N : capacity and hit time, separated by a space
$N + 2$	Main memory access time, in cycles
$N + 3$	M , the number of accesses in the trace
$N + 4$	M block IDs, separated by spaces, in the order they are requested

7 Output format

For each access, print a line in this form, using one-based numbering for the access and naming the level that served it as L1, L2, and so on, or Main Memory:

```
Access i: block <id> -> served by <level>, cost <cost> cycles
```

After all accesses, print a blank line, then a summary in this form, listing every cache level in order followed by main memory:

```
Summary:
  L1 hits: <count>
  L2 hits: <count>
  ...
Main Memory hits: <count>
Total cycles: <total>
Average cycles per access: <average, two decimal places>
```

8 Sample input and output

The sample below uses two cache levels. Level 1 (L1) holds up to 2 blocks with a hit time of 1 cycle. Level 2 (L2) holds up to 4 blocks with a hit time of 8 cycles. Main memory takes 100 cycles.

Table 2: Sample cache configuration used below

Level	Capacity (blocks)	Hit time (cycles)
L1	2	1
L2	4	8

Contents of `input.txt` for this sample:

```
2
2 1
```

```
4 8
100
9
5 7 5 9 5 7 12 5 9
```

Expected console output:

```
Access 1: block 5 -> served by Main Memory, cost 109 cycles
Access 2: block 7 -> served by Main Memory, cost 109 cycles
Access 3: block 5 -> served by L1, cost 1 cycles
Access 4: block 9 -> served by Main Memory, cost 109 cycles
Access 5: block 5 -> served by L1, cost 1 cycles
Access 6: block 7 -> served by L1, cost 1 cycles
Access 7: block 12 -> served by Main Memory, cost 109 cycles
Access 8: block 5 -> served by L1, cost 1 cycles
Access 9: block 9 -> served by L2, cost 9 cycles
```

```
Summary:
  L1 hits: 4
  L2 hits: 1
  Main Memory hits: 4
  Total cycles: 449
  Average cycles per access: 49.89
```

A second, smaller test case is included with the files for this lab so you can check your program against expected output before you submit. Since your program always reads from a file named `input.txt`, test one case at a time: copy or rename the sample file you want to try to `input.txt` in your project's working directory, run your program, and compare the console output to what is shown above.

A third file, `sample_input_3.txt`, is also included. It uses three cache levels instead of two, along with a longer trace, and no expected output is given for it. Use it to check that your program generalizes correctly to any number of cache levels, not just the two-level case shown above, and to see the L2 and L3 hit cases play out.

9 Walkthrough of the sample trace

It can help to see exactly what the recursion is doing before you write any code. The two examples below walk through the calls and returns by hand for two accesses from the sample trace above, and the table after them shows what happens to both caches across the entire trace.

Access 1: block 5 (a full miss, cascading down and filling both levels on the way back up)

Both caches start empty. Conceptually, `accessLevel(5, 0)` checks L1's contents, which are empty, and does not find block 5, so it recurses to `accessLevel(5, 1)`. That call checks L2's contents, also empty, and does not find block 5 either, so it recurses again, to `accessLevel(5, 2)`. Since there are only two cache levels here, level 2 is past the last one, which is the base case: the request is served by main memory, which always succeeds, at a cost of 100 cycles. That 100 is returned back up to the call at level 1. Only now, after that call has returned, does level 1 do its own work: it adds its own hit time, 8 cycles, to the 100 it received, for a running total of 108, and it caches block 5 in L2 since L2 has room. That 108 is returned back up again, to the call at level 0. Level 0 does the same thing: it adds its own hit time, 1 cycle, for a total of 109, and caches block 5 in L1

since L1 also has room. That final value, 109, is what gets printed as the cost of the first access, and it is only computed after all three calls have returned. Both caches now contain block 5.

Access 9: block 9 again (an L2 hit, with L1 already full)

Now skip ahead to the last access, another request for block 9. By this point, L1 holds blocks 5 and 7, and L2 holds blocks 5, 7, 9, and 12, both full. `accessLevel(9, 0)` checks L1's contents and does not find block 9 there, since L1 filled up before block 9 ever had a chance to be cached, back at access 4. It recurses to `accessLevel(9, 1)`, which checks L2's contents and does find block 9 there this time, so this call returns immediately with L2's hit time, 8 cycles, without recursing any further. Back at level 0, that 8 is combined with L1's own hit time of 1 cycle, for a total of 9 cycles. Level 0 would normally cache the block now that it knows where it was found, but L1 is already at its capacity of 2, so the insertion is skipped and L1 is left unchanged. This is the case referenced in the thinking question earlier in this handout: block 9 does get faster on its second request, but only down to 9 cycles rather than the 1 cycle a request for block 5 or 7 gets, because L1 never had room to cache it.

Table 3: Cache contents and cost for every access in the sample trace

Access	Block	L1 (before → after)	L2 (before → after)	Served by	Cost
1	5	{ } → {5}	{ } → {5}	Main	109
				Memory	
2	7	{5} → {5,7}	{5} → {5,7}	Main	109
				Memory	
3	5	{5,7} (no change)	{5,7} (no change)	L1	1
4	9	{5,7} (no change)	{5,7} → {5,7,9}	Main	109
				Memory	
5	5	{5,7} (no change)	{5,7,9} (no change)	L1	1
6	7	{5,7} (no change)	{5,7,9} (no change)	L1	1
7	12	{5,7} (no change)	{5,7,9} → {5,7,9,12}	Main	109
				Memory	
8	5	{5,7} (no change)	{5,7,9,12} (no change)	L1	1
9	9	{5,7} (no change)	{5,7,9,12} (no change)	L2	9

10 What to submit

Submit your work by committing your `labw03` project to your repository for this assignment and pushing that commit before the deadline. Use the file name given in the Requirements section above for your source file, and make sure your project builds and runs correctly, whether you are working from the command line or an IDE, before you push it.

11 Resources

- TechSpot, "[Explainer: L1 vs. L2 vs. L3 Cache](#)", for background on why real processors use more than one cache level.

-
- University of Toronto CSC148 course notes, "[Recursion and the call stack](#)", for a review of how recursive calls and returns work if you want a refresher before starting.
 - UC Berkeley CS 61C course notes, "[Average Memory Access Time \(AMAT\)](#)", which shows the same nested formula used to analyze real multi-level cache performance. This is the standard method used to reason about cache designs, not a claim that the hardware itself executes recursive function calls.
-

A note on these resources

These links were found with help from an AI system and web search, not from the instructor's own prior use. They look relevant, but none have been personally verified by the instructor. Treat them as a starting point, and let your instructor know if one is broken, off-topic, or wrong.
